IKS

PAGE NO.:
DATE: / /

Module 1

Chapter 1 → 2|12-1

Concept of Sastra as Formal knowledge System.

In Indian knowledge Tradition, a Sastra is a systematically organized body of knowledge governed by clearly defined principles, rules, methods, classifications, and validation procedures. It is not merely a collection of information but a formal framework that specifies:

i) What constitutes valid knowledge.
ii) How knowledge should be organized
iii) How reasoning should be performed
iv) How conclusions should be verified.

A Sastra operates much like a modern formal system in Computer Science, where rules and structures govern the ~~represent~~ representation and processing of information.

Characteristics of ~~Sastra~~ Sastra :

i) Formal ~~SE~~ Structure
ii) Rule - Based Processing : Operations are governed by explicit rules rather than intuition.
iii) Consistency : The system avoids contradictions and maintains logical coherence.

iv) Reproducibility : Different practioners applying the same rules obtain the same result.

v) Validation Mechanism : Methods exist to verify correctness and reliability.

Sutra {सूत्र} Method as Compressed Symbolic Encoding

One of the most remarkable methodologies of Indian knowledge systems is the Sutra style. A Sutra is an extremely concise statement containing maximum information in minimum words.

Traditional characteristics include :

- Brevity
- Non-ambiguity
- Precision
- High information density.

The purpose was efficient storage, memorization, transmission, and processing of knowledge.

Ex : Panini's Astadhyayi contains approximately 4,000 Sutras (सूत्र) that generate the entire grammar of Sanskrit through rule combinations.

A small rule can generate thousands of valid linguistic forms.

PAGE NO.:
DATE: / /

Pramana theory

Pramana means a ~~valid mea~~ source of knowledge. In Indian knowledge Systems (IKS), Pramana Theory explains how valid and reliable knowledge is obtained.

The word Pramana is derived from Sanskrit and refers to the process or method through which a person obtains correct knowledge (Prama).

Main types of Pramana

Different schools of Indian philosophy recognize different Pramanas. The ~~s~~ important Pramanas are :

1) Pratyaksa - Direct Observation.

knowledge obtained through direct preception or observation.

Traditional example: Seeing smoke rising from a hill.

Computer Science Example: A sensar ~~reports~~ reports: Temperature = 95°C

This reading comes directly from observation.

Ex: Camera Input, Temperature sensor, keyboard Input, Network packet capture.

2) Anumana :— Inference.

knowledge derived from reasoning based on observed evidence.

Traditional example: Smoke is observed,
therefore:
Fire is present.

Computer Science Example:
if: CPU Usage = 100%
Response time = High.
then infer:
Server Overload.
The overload is not directly observed but inferred.

3) Agama — Authoritative knowledge.

Knowledge ~~are~~ accepted from a trusted authority or reliable source.

Traditional Example: learning astronomy from a recognized scholar.

Computer Science Example: A software engineer trusts
• Official documentation • Standard Specification
• Verified database.

PAGE NO.:
DATE: / /

Importance in Computational Systems

Pramana Theory is relevant to computational systems and AI because it provides a framework for understanding different sources and methods of acquiring knowledge. Perception can be related to sensors/data, inference to reasoning and AI algorithms, comparison to pattern matching; and testimony to knowledge obtained from reliable sources.

Knowledge Validation and Structured Reasoning.

A knowledge system becomes useful only when information can be validated and reasoned about systematically. Indian logical traditions developed sophisticated reasoning frameworks to ensure correctness.

→ Structured Reasoning Process

Step 1 : Observation.
Collect evidence.
Ex :
CPU Temperature = 100°C.

Step 2: Rule Application.
Apply known rules; IF temperature > 90°C
then overheating.

Step 3: Inference
Drive conclusion. ; system is overheating.

Step 4: Validation
Check whether conclusion matches additional evidence ;
Fan speed increased.
Performance reduced.
Conclusion ~~conf~~ confirmed.

Knowledge Representation.
Representation of knowledge in machine-readable form.
Examples: > Ontologies > knowledge Graphs.
> Semantic Networks > Databases.

Chapter 2

PAGE NO.:
DATE:

Pingala's Chandahsastra as Binary Encoding and Combinatorial Generation

One of the most remarkable contributions of ancient Indian knowledge systems to computational thinking is found in Pingala's Chandahsastra, a treatise on Sanskrit prosody. Composed around 2nd century BCE (or earlier according to some scholars), the text studies patterns of Laghu (short syllables) and Guru (long syllables) in poetic meters.

What makes Pingala's work especially significant for Computer Science is that it contains methods equivalent to:

- → Binary Encoding
- → Recursive Enumeration
- → Search and Retrieval Algorithms.
- → Tree Structures
- → Combinatorial Mathematics.
- → Pascal's Triangle
- → Algorithmic Generation.

PAGE NO.
DATE: / /

Laghu - Guru Representation as Binary Encoding.

The foundation of Pingala's system is the representation of poetic syllables as :-

Syllable Type	Meaning
Laghu	Short syllable
Guru	Long syllable

Each verse pattern can be represented as a sequence of Laghu and Guru syllables:

> Binary Interpretation

In computer science, binary encoding uses:
0 and 1

Similarly :-

Chandah sastra	Binary system
Laghu	0
Guru	1

> Computer Science Relevance :

Modern computers represent & using only binary :-

- Numbers
- Characters.
- Images
- Programs.

Ex:- Character A : 01000001.
Character B : 01000010.

In the system, complex information is encoded through symbols.

PAGE NO.:
DATE: / /

Prastara {Systematic enumeration of pattern}

Prastara is an ancient indian method for the systematic enumeration of all possible patterns or combinations. It is mainly associated with Pingala's Chandasastra, where it was used to systematically list all possible arrangements of Laghu (short) and Guru (long) syllables in Sanskrit poetic metres.

- Importance in Computational Systems.

Prastara has a strong connection with modern computer science and computational systems:

i) It is similar to binary enumeration, where each position has two possible values (0 or 1)

ii) It can be used to generate all possible combinations or patterns.

iii) The number of patterns for n binary positions is 2^n.

iv) It demonstrates an early form of algorithmic and systematic thinking.

v) The concept is related to modern combinatorics, pattern generation, permutations, and enumeration algorithms.

Ex:- Two syllables :- L & G.
Possible combinations :- LL, LG, GL, GG.
Total :- $2^2 = 4$.

Nasta and Uddista {Index retrieval mechanisms}.

1] Nasta {Pattern - Position}.

It is the process of finding the position or number of a particular pattern when the pattern itself is known but its position in the complete Prastara is unknown.

Ex:- If all binary patterns of a given length are systematically arranged, Nasta helps us determine the index of a given pattern.

2] Uddista (Position → Pattern)

It is the reverse process of Nasta, Here the position or number of a pattern is given, and we determine the actual pattern corresponding to that position.

Diff b/w Nasta & Uddista.

Nasta	Uddista.
> Pattern is given	> Position is given
> Finds the positional/rank	> Finds the pattern.
> Pattern → Number	> Number → Pattern
> Similar to encoding/indexing	> Similar to decoding.

PAGE NO.
DATE: / /

Importance in Computer Computational System

Nasta and Uddista demonstrate an early form of algorithmic thinking. Their concepts are closely related to modern computational operations such as :

1) Binary number
2) Indexing
3) Encoding and decoding
4) Pattern Generation
5) Combinatorial enumeration.

Meru - Prastara and Combinatorial Expansion.

Meru-Prastara is an ancient Indian mathematical method for arranging numbers in a triangular pattern to calculate combinations. It is closely associated with the work of Pingala and later mathematicians such as Halayudha.

It is also known as the Meru arrangement and is closely related to what is called Pascal's Triangle in modern mathematics.

Structure of Meru - Prastara

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
1   5   10  10  5   1
```

Each inner number is obtained by adding the two numbers directly above it.

Combinatorial Expansion

Meru-Prastara can be used to find the coefficients in the expansion of expressions such as: $(a+b)^n$

For example:
$(a+b)^2 = a^2 + 2ab + b^2$
The coefficients 1, 2, 1 come from the third row of Meru-Prastara.

Importance in Computational Systems

Meru-Prastara is relevant to computational systems because it demonstrates:

i) Systematic generation of numerical patterns.
ii) Calculation of combinations and binomial coefficients.
iii) A recursive method where each value depends on previous values.
iv) Applications in combinatorics, probability and algorithm design.
v) A historical connection between ancient Indian mathematics and modern Pascal's Triangle.

chapter 3

PAGE NO.:

Panini's Astadhyayi as a Generative Formal Grammar

Panini Astadhyayi is an ancient Sanskrit grammatical system created by the Indian grammarian Panini. It consists of approximately 4000 concise grammatical rules (sutras) arranged into 8 chapters.

It can be viewed as a generative formal grammar because it rules systematically take basic linguistic elements and generate valid sanskrit words and sentences.

Connection with Formal Grammar:

Panini's system has several characteristics similar to modern formal grammars and computational systems:

i) Rule-Based: Grammar is represented through precise rules.

ii) Generative: Rules can generate valid linguistic forms from basic elements.

iii) Formal and concise: The sutras use a compact system of technical symbols and conventions.

iv) Context-sensitive operations: The application of some rules depends on surrounding linguistic elements.

v) Algorithmic Nature: Rules can be applied step-by-step to derive the final form.

Importance in Computational systems:

Panini's grammar is relevant to Nature Language Processing (NLP) and computational linguistics because it demonstrates how a complex language can be represented using a finite and systematic set of rules. Its ideas have inspired research in rule-based NLP, grammar parsing, morphological analysis and language generation.

Rule-Based Generative Structure

A Rule-Based Generative Structure is a system in which a set of predefined rules is used to generate valid outputs from given inputs.

The central idea behind Panini's grammar is that language can be generated from a finite set of rules.
Instead of listing every possible Sanskrit word, Panini created rules that produce infinitely many valid expressions.

This is exactly how modern programming languages and formal grammar operates.

Generative Process :-

Input

↓

Rule Application

↓

Phonological Transformation

↓

Output.

Meta-Rules and Rule Precedence.

In Panini's grammatical system, meta-rules are rules that explain how other grammatical rules should be interpreted or applied. They do not directly generate words; instead, they control the application, interpretation, and interaction of grammatical rules.

Rules ~~precen~~ precedence refers to determining which rule should be applied when two or more rules are applicable at the same time.

1] Meta-Rules

Meta-rules act like "rules about rules". They provide instructions for handling the large number of grammatical rules in the Astadhyayi.

They help determine :

i) When a rule should be applied.

ii) Which rules are applicable in a particular situation.

iii) How conflicting rules should be handled.

iv) The scope and interpretation of other rules.

2] Rule Precedence

Sometimes, multiple rules may apply to the same input. Applying all of them arbitrarily could produce an incorrect result. Therefore, Panini's system contains principles for deciding which rule gets priority.

A simplified representation is :-

Input → Applicable Rules → Resolve Conflict

↓

Output ← Apply Higher-Priority rule

One important principle is "Viprathisedhe param karyam", meaning that when two rules conflict, the later rule is generally applied.

Example:

```
if x > 0:
    print("Positive")
elif x > 10:
    print("Large Positive")
```

If x = 15 then both conditions are true, The precedence structure determines which executes first.

Context - Sensitive Operations.

Context - sensitive operations are operations in which the result of applying a rule depends on the surrounding context or conditions of the input.

In the context of Panini's Astadhyayi, a grammatical rule may produce a particular transformation only when specific conditions are satisfied. Therefore, the same word or linguistic element may be treated differently depending on what occurs before or after it.

Key Feature:

1) Depends on context: The rule considers surrounding elements.

2) Condition-based: A transformation occurs only when specified conditions are met.
3) Systematic: The operation follows predefined grammatical rules.
4) Avoids ambiguity: Context helps determine the correct transformation.
5) Generative: Different contexts can produce different valid outputs.

Connection with Computational systems

Context-sensitive operations are important in modern computer science:

i) NLP: A word's meaning or grammatical form can depend on surrounding words.
ii) Compilers: Code transformations depends on the surrounding syntax and context.
iii) Parsing: The interpretation of a symbol can depend on its position and neighbouring symbols.
iv) Pattern matching: Different outputs can be produced depending on the surrounding pattern.
v) Rule-based AI: Rules are activated when particular conditions are satisfied.

Ex: IF A occurs before B
THEN change A to C.

This demonstrates that the operation is context-sensitive because the output depends not only on A, but also on what surrounds it.

PAGE NO.:
DATE: / /

Conflict Resolution Mechanisms.

In a large rule system, multiple rules may compete for application. A robust formal system requires mechanisms for resolving conflicts: Panini developed sophisticated strategies for determining which rule should be executed.

Paninian Techniques :

- Rule Ordering : Later rules may override earlier rules.
- Specificity Principle : More specific rules override general rules.
- Domain Restriction : Rules apply only within defined scopes.

Example : Arithmetic Parsing.

Expression : $2 + 3 \times 4$.

Possible Interpretations : $(2+3) \times 4$.

OR

$2 + (3 \times 4)$

Precedence rules choose :

$2 + (3 \times 4)$

Similarly, Panini's conflict resolution mechanisms ensure a unique grammatical output.

Context-Free Grammar (CFG)

A context-free grammar is a formal system used to describe and generate the structure of sentences or strings in a language. It consists of a set of rules where a single non-terminal symbol can be replaced by a sequence of terminals and/or non-terminals regardless of its surrounding context.

In computational systems, CFGs are mainly used in formal languages, compiler design, pasing, and Natural Language Processing (NLP).

A CFG is represented as :

$$G = (V, \Sigma, P, S)$$

where :

- $V \rightarrow$ set of non-terminal symbols.
- $\Sigma \rightarrow$ set of terminal symbols.
- $P \rightarrow$ Set of production rules.
- $S \rightarrow$ Start symbol.

It is called context-free because a production rule can be applied to a non-terminal without considering the symbol surrounding it.

Ex : $NP \rightarrow N$.

can be applied whenever NP occurs, regardless of what comes before or after it.

PAGE NO:
DATE: / /

Connection with IKS :-
Panini's Astadhyayi is often discussed in relation to formal grammar because it provides a highly systematic rule-based description of Sanskrit. However, Panini's grammar should not simply be equated with a modern CFG; its rules include context-dependent operations and sophisticated rule interaction mechanisms.

Rewrite Systems

- A Rewrite system is a formal system in which symbols or structures are repeatedly replaced or transformed according to a set of predefined rules.

In IKS and Computational Systems, rewrite systems are relevant to the study of Panini's Astadhyayi, ~~be a~~ because Paninian grammar uses rules that transform linguistic elements step-by-step to generate valid Sanskrit words.

- Rewrite Systems in Panini's Grammar :-

In the Astadhyayi, grammatical elements undergo various transformations through rules.

They can ~~not~~ include :-

- > Addition of suffixes.
- > Replacement of linguistic elements.
- > Sound transformations.
- > Combination of elements.
- > Removal or modification of elements.

Automata theory

Automata Theory is a branch of computer science that studies abstract mathematical machines (automata) and how they process ~~into~~ input according to a set of rules.

An automaton reads an input string symbol by symbol, changes it state according to predefined rules, and finally determines whether the input is accepted or rejected.

In IKS and Computational Systems, Automata Theory can be connected with the systematic, rule-based nature of Panini's Astadhyayi, where linguistic forms are processed through a sequence of grammatical rules.

Basic Components:

A simple automaton generally consists of:

i> state :- Different conditions or stages of the machine.

ii> Input symbols :- Symbols given to the machine.

iii> Transition rules :- Rules that determine how the machine changes state.

iv> Initial state :- The state where processing begins.

v> Final / Accepting states :- States indicating that the input is accepted.

~~May~~ Major Types of Automata :-

> Finite Automata
> Pushdown Automata
> Turing Machine.

Parsing Algorithms

Parsing is the process of analyzing an input string or sentence according to a given grammar to determine its structure.

A parsing algorithm is a systematic procedure that checks whether an input follows the rules of a grammar and, if valid, constructs its parse tree or syntactic structure

In IKS and Computational Systems, parsing is relevant to Panini's Astadhyayi because

Sanskrit grammatical rules can be systematically applied to analyze and derive linguistic structures.

Foundations of ~~NLP~~ NLP

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, process, analyze and generate human language.

The foundations of NLP consists of the linguistic and computational techniques required to convert human language into a form that computers can process.

~~#~~ Applications of NLP
- > Machine translation.
- > Chatbots and Virtual Assistants
- > Speech Recognition
- > Sentiment Analysis
- > Text Summarization
- > Information Retrieval
- > Question Answering.

Connection with IKS:-
Indian ~~lg~~ linguistic traditions, particularly Panini's Astadhyayi, provide a systematic rule-based approach to language. Concepts such as grammar, word formation, rule application